Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)		
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

Chemistry assistant as listed in the above text

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

chemicalequationbalancer.com and webqc.org have similar function

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

Users input an equation and starting/ending masses and the machine computes the intermediate steps to determine properties of the reaction. It's meant to save time for people who already know how to do it manually.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The program will save time by doing trivial calculations for the user.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

A valid chemical reaction that at least has the same elements on both sides. Optionally a measured amount of reactant or product.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

It will read over both sides of the reaction and systematically multiple reaction coefficients until both sides have the same total amount of atoms. Then it will divide if there is a common factor using modulo operations or similar.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

A balanced reaction and some numbers for grams of reaction/product etc

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Specific atoms like Sulfur or Carbon.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

I need reactant mass and element/molecule formulas. I don't need what medium the reaction took place or what temperature etc. (physical conditions)

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Figuring out what letters are part of an element because some are one character long and others are two. The ones with two letters always have the second letter lowercase but it would be hard to make sure users always input the correct capitalization. Also drafter will be difficult to link up to the logic.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I don't want to use a LLM so I will do most of my coding over the weekend. ~1.5 hours Saturday and ~2.5 hours Sunday

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

I want to make a program that is intuitive to use and learn how to create a drafter website from scratch, without a drafter outline already being given to me.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a web app that will take in unbalanced chemical equations, balance them and show mass of products based on inputted reactant mass. What are a few common problems I could encounter doing this. Also what features do similar web tools lack that I could think about adding.

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

It warned that polyatomic ions may break a program and also suggested adding feedback when user input is wrong telling them why it is wrong rather than doing nothing.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Equation autocomplete seems too complicated for an introductory python student. What's an alternative feature that would still be cool but simpler to code.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

The AI suggests omitting autocomplete and showing step-by-step instructions so users understand how the website got to the answer instead.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I anticipate using drafter documentation since I'm not very familiar with it. If I get stuck on the python code I will use python documentation.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

First tell the AI the error and ask to fix it. If that fails I would investigate the error and find similar keywords on drafter/bakery documentation to see how its intended use.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

This is actually a really interesting web app idea, especially for a general chemistry audience. I would think of it as more than just a chemical equation balancer: a stoichiometry calculator that takes a student from an unbalanced reaction all the way to theoretical product yield.

There are three main areas where you could run into problems:

1. Correctly interpreting and balancing chemical equations.
2. Performing stoichiometric calculations accurately.
3. Designing the app so that it handles real chemistry problems rather than only simple textbook examples.

I'll break down the technical challenges first, then some features that could make your app stand out.

1. Common problems you could encounter

A. Balancing chemical equations

This is probably the first major challenge.

1. Different equations have different levels of complexity.

For example, these are very different problems:

- Simple: $H_2 + O_2 \rightarrow H_2O$
- Combustion: $C_3H_8 + O_2 \rightarrow CO_2 + H_2O$
- Polyatomic ions: $Al_2(SO_4)_3 + Ca(OH)_2 \rightarrow Al(OH)_3 + CaSO_4$
- Redox reactions: $MnO_4^- + Fe^{2+} \rightarrow Mn^{2+} + Fe^{3+}$ (in acidic solution)

A balancing algorithm that works for simple reactions might fail when it encounters polyatomic ions, charges, or redox reactions.

Potential solution: Use a matrix-based balancing algorithm. Represent each element's atom counts as a system of linear equations, then solve for the smallest whole-number coefficients.

2. Chemical formulas are difficult to parse.

Your app needs to distinguish between:

- $Ca(OH)_2$ — parentheses and subscripts.
- $Al_2(SO_4)_3$ — nested atom counts.
- NH_4NO_3 — multiple occurrences of the same element.
- Fe^{3+} — ionic charge, not an atom count.

If you build your own formula parser, this could be a significant source of bugs.

3. Some reactions have multiple valid-looking solutions.

A chemical equation can have coefficients that are mathematically proportional. For example:

$$2H_2 + O_2 \rightarrow 2H_2O$$

and

$$4H_2 + 2O_2 \rightarrow 4H_2O$$

Both are balanced, but you generally want the smallest whole-number coefficients.

You need to normalize the coefficients using their greatest common divisor.

B. Calculating product mass

Once the equation is balanced, the next challenge is converting reactant mass into product mass.

4. Limiting reactants

This is one of the biggest potential problems.

If someone inputs the mass of two or more reactants, you cannot simply calculate the product from whichever reactant they entered first.

You need to:

1. Convert each reactant's mass into moles.
2. Divide each mole quantity by its balanced coefficient.
3. Identify the smallest resulting value.
4. Use that limiting reactant to calculate theoretical product yield.

If you only accept one reactant mass, your app should clearly communicate that it assumes the selected reactant is the limiting reactant.

5. Molecular weight calculations

Your app needs an accurate periodic table database.

For example, the molecular weight of:

$$Ca(OH)_2$$

is calculated using:

$$40.078 + 2(15.999) + 2(1.008)$$

You need to account for parentheses, subscripts, and repeated elements correctly.

6. Significant figures and rounding

This is an easy detail to overlook.

For example, if someone inputs 2.5 grams of a reactant, your app shouldn't necessarily report a product mass of 12.483729 grams.

Consider:

- Preserving the precision of user inputs.
- Applying significant figure rules.
- Allowing users to choose decimal places.
- Keeping full precision internally and rounding only displayed results.

C. User input and error handling

7. Users might enter chemically invalid reactions.

Examples:

- Misspelled element symbols.
- Missing products.
- Incorrect subscripts.
- Invalid chemical formulas.
- Incorrect charges.

Instead of just displaying an error, your app should tell the user what went wrong.

8. Chemical equations can be ambiguous.

For example, a reaction might have different products depending on the reaction conditions.

An equation balancer cannot necessarily determine what reaction actually occurs just from the reactants.

Your app should distinguish between:

- Balancing a reaction that the user supplies.
- Predicting the products of a reaction.

I'd strongly recommend starting with the first. Predicting products introduces a much larger chemistry problem.

2. What similar tools offer (and where you can improve)

I looked at several existing chemical equation balancers and stoichiometry calculators. One thing I noticed is that the market has already moved beyond simply balancing equations.

Some tools already offer:

- Automatic equation balancing.
- Molecular weight calculations.
- Limiting reactant identification.
- Theoretical yield calculations.
- Excess reactant calculations.
- Step-by-step solutions.
- Atom-count verification tables.

For example, Tembrica Chemistry Calculator offers worked stoichiometry solutions and a balancing trainer. ScienLytic offers an atom-count ledger, limiting reactant analysis, and theoretical and actual yield calculations. EvvyTools includes equation balancing, stoichiometry, and printable worksheets.

Tembrica+2

This means I'd avoid making your app just another calculator. Instead, I'd focus on making it more useful for understanding and solving chemistry problems.

Here are some features I'd consider.

A. Educational features

My strongest recommendation

1. Show the entire calculation process

Rather than simply displaying:

$$36.0 g H_2O$$

Show the entire process:

1. Identify the given mass.
2. Convert grams of reactant to moles.

3. Use the balanced equation's mole ratio.
4. Convert moles of product to grams.
5. Display the final answer.

For example:

$$4H_2 + 2O_2 \rightarrow 4H_2O$$

If a student enters 4.00 g of hydrogen and has excess oxygen, show:

$$4.00gH_2 \times \frac{1molH_2}{2.016gH_2}$$

$$\times \frac{2molH_2O}{2molH_2} \times \frac{18.015gH_2O}{1molH_2O}$$

This is much more useful to someone learning stoichiometry than a calculator that only provides the answer.

2. Multiple explanation levels

Let users select how much explanation they want.

- Quick Answer: Balanced equation and product mass.
- Show Work: All conversions and mole ratios.
- Teach Me: Explain why each calculation is performed.

This would appeal to both students who just want to check their work and students learning the material.

3. Interactive balancing practice

Instead of immediately balancing the equation, let users try it themselves.

For example:

\[\boxed{?}H_2+\boxed{?}O_2\rightarrow\boxed{?}H_2O \]

Users enter coefficients, and the app checks their answer.

You could even explain which element is currently unbalanced.

4. Explain mistakes instead of just marking answers wrong

Imagine a student enters:

$$H_2 + O_2 \rightarrow H_2O_2$$

Your app could explain that balancing requires changing coefficients, not subscripts.

Other possible explanations:

- You used the wrong mole ratio.
- You converted grams to moles incorrectly.
- You used the excess reactant instead of the limiting reactant.
- Your equation is not balanced for oxygen.

This is a particularly useful educational feature.

B. Calculation features

5. Multiple reactant inputs

I'd consider this essential.

Allow users to enter the mass of every reactant rather than just one.

For example:

Reactant	Input mass
N_2	28 g
H_2	6 g

Then display:

- Limiting reactant.
- Excess reactant.
- Amount of excess reactant remaining.
- Theoretical mass of every product.

6. Reverse stoichiometry calculator

Most basic calculators work forward:

Reactant mass → Product mass.

You could also allow users to enter a desired product mass and calculate how much of each reactant would be required.

This is especially interesting from an engineering perspective.

7. Actual yield and percent yield

Allow users to enter an experimental product mass after calculating theoretical yield.

Then calculate:

\[\%\text{Yield}= \frac{\text{Actual Yield}}{\text{Theoretical Yield}}\times100 \]

You could display theoretical and actual yield side by side.

8. Multiple units

Don't limit the calculator to grams.

Consider:

- Grams
- Kilograms
- Milligrams
- Moles
- Millimoles
- Number of particles

You could also eventually support solution concentrations and gas volumes.

C. Interface and usability features

9. Visual representation of the reaction

Show a visual representation of how much reactant is consumed and how much product is produced.

For example:

Illustrative reaction visualization

N_2

Reactant

28 g

NH_3

Product

34 g

Simplified illustration: assumes sufficient hydrogen and uses rounded masses.

The visualization could update automatically as users change their reactant amounts.

10. Mass conservation checker

This could be a particularly nice feature.

Show the total mass of reactants consumed and the total theoretical mass of products formed.

For example:

$$m_{reactantsconsumed} = m_{productsformed}$$

You could display a green check when the calculated values agree within numerical precision.

This helps students connect stoichiometry with the law of conservation of mass.

11. Save and export calculations

Allow users to:

- Copy the balanced equation.
- Download a PDF of the calculation.

- Save previous reactions.
- Export the step-by-step work.
- Share a calculation with a classmate.

12. Mobile-friendly equation input

Chemical formulas are awkward to type on a phone.

You could add a chemical formula input interface with:

- Subscript buttons.
- Superscript/charge buttons.
- Parentheses.
- Reaction arrows.
- A searchable periodic table.

3. Features that could really differentiate your app

These are ideas I'd consider if you want your app to be more than a combination of existing calculators.

Feature	Why it's interesting
Equation input autocomplete	Suggests valid chemical formulas as users type.
Reaction history	Keeps a record of previous calculations.
Automatic error detection	Explains why an equation cannot be balanced.
Step-by-step learning mode	Teaches the underlying chemistry.
Adjustable significant figures	Makes answers appropriate for different assignments.
Reaction visualization	Helps students understand mole ratios visually.
Mass conservation visualization	Demonstrates conservation of mass.
Target product calculator	Works backward from desired product mass.
Homework checking	Lets students enter their own work and identify mistakes.

mode	
Reaction condition notes	Lets users document temperature, pressure, and other relevant conditions.

One important observation: some competitors already have features like atom ledgers, reaction visualization, and error explanations. So don't assume these are completely new. I'd focus on making the features work together particularly well.

Apexa Pro+2

4. How I'd approach building it

I would avoid trying to implement everything immediately.

Here's how I'd break the project into stages.

Phase 1 — Core functionality

Start here

- Equation input and parsing.
- Automatic equation balancing.
- Molar mass calculations.
- Single-reactant mass-to-product calculations.
- Display balanced equation and final product masses.

Phase 2 — Full stoichiometry calculator

- Multiple reactant inputs.
- Limiting reactant identification.
- Excess reactant calculations.
- Theoretical yield.
- Actual yield and percent yield.
- Significant figures.

Phase 3 — Educational features

- Step-by-step calculations.
- Explanation of mole ratios.
- Interactive balancing practice.
- Error explanations.
- Mass conservation verification.

Phase 4 — Additional functionality

- Reverse stoichiometry.
- Additional units.
- Reaction history.
- PDF export.
- Reaction visualizations.

5. Technical considerations

A few things I'd recommend from a development standpoint:

Component	Suggested approach
Frontend	React
Styling	Tailwind CSS
Equation balancing	Matrix / null-space algorithm
Formula parsing	Dedicated chemical formula parser
Atomic masses	Periodic table data stored locally
Calculations	JavaScript
Storage	Local storage initially

One particularly important technical recommendation: use exact rational arithmetic for equation balancing rather than floating-point approximations.

The balancing algorithm can construct a matrix of element counts and solve for its null space. Convert the resulting coefficients into the smallest positive whole-number ratio.

Also, build a test suite containing simple equations, nested parentheses, hydrates, ionic equations, and reactions with no valid balancing solution. Verify every result independently by checking atom counts and total charge.

For an initial version, I'd limit the supported chemistry to conventional molecular equations before expanding into ionic and redox reactions.

6. My biggest recommendation

If I were building this project, I'd make the central experience look something like this:

Stoichiometry Calculator

Concept

Enter a reaction to balance and calculate product yield.

CHEMICAL EQUATION

$H_2 + O_2 \rightarrow H_2O$

Balanced successfully

$2H_2 + O_2 \rightarrow 2H_2O$

Reactant amounts

Hydrogen (H_2)

4.00 g

Oxygen (O_2)

Excess

Calculated results

THEORETICAL PRODUCT MASS

35.74 g

Water (H_2O)

Limiting reactant

H_2

Moles of H_2

1.984 mol

Moles of H_2O

1.984 mol

Show calculation

Conceptual UI mockup — illustrative calculations assume sufficient oxygen.

The thing I'd really emphasize is making the transition from equation balancing to stoichiometry seamless. A student shouldn't have to copy their balanced equation into a separate calculator.

My top three priorities would be:

1. Accuracy: Handle chemical formulas and calculations correctly.
2. Transparency: Show exactly how the answer was calculated.
3. Usability: Make entering a reaction and getting a result as straightforward as possible.

That combination would make a genuinely useful tool for introductory chemistry students and potentially engineering students beyond general chemistry.

A possible next step: Design the actual interface before coding it. I'd start with a clean, modern chemistry calculator with an equation input at the top, reactant mass inputs underneath, and calculated product yields alongside a step-by-step solution.

Can you design a visual mockup of the web app?

I would actually skip equation autocomplete for now. As an introductory Python student, you can add some really interesting features without having to build complicated chemical formula recognition or natural-language processing.

I'd focus on features that use basic Python concepts like loops, dictionaries, functions, conditionals, and lists.

Here are some ideas I'd recommend, ranked by how useful and achievable they are.

1. Interactive step-by-step solution generator

Best overall choice

Instead of just giving users the product mass, have Python generate the individual steps used to calculate it.

For example, if someone enters 4.00 g of H_2 and has excess O_2:

The app displays:

1. Balanced equation: $2H_2 + O_2 \rightarrow 2H_2O$
2. Convert grams H_2 to moles H_2.
3. Apply the mole ratio.
4. Convert moles H_2O to grams H_2O.
5. Display theoretical yield.

Why it's achievable: Each calculation can be its own Python function. You can store the steps in a list and display them one at a time.

You could even add a button that says "Why am I doing this?" to explain each step.

2. Reaction difficulty selector

Let users choose the type of problem they want to practice.

Choose a reaction type



Basic

Simple synthesis



Combustion

Combustion reactions



Decomposition

Decomposition reactions



Limiting reagent

Multiple reactants

You could have a collection of predefined chemical equations stored in Python dictionaries.

When a user selects a category, randomly select an equation from that category.

Why it's achievable: This mainly uses dictionaries, lists, random selection, and conditionals.

3. Limiting reactant visualizer

This would be one of the coolest features for an introductory chemistry app.

Instead of just telling someone which reactant is limiting, visually show how much of each reactant is available relative to the balanced equation.

For example:

$$N_2 + 3H_2 \rightarrow 2NH_3$$

Suppose a user has:

- 2 moles of N_2
- 3 moles of H_2

Your app could display:

Reactant availability

N_2

2 mol available

H_2

3 mol available

H_2 is limiting

Result

2 mol NH_3

One mole of N_2 remains unused.

Why it's achievable: You only need to calculate the available mole ratios and display them using simple bars.

4. Significant figures assistant

This is a smaller feature, but very useful for chemistry students.

Allow users to:

- Enter their measured reactant mass.
- Select the number of significant figures.
- Choose how many decimal places to display.
- See the unrounded and rounded answers.

You could even explain why the final answer has a particular number of significant figures.

Why it's achievable: Python has built-in formatting tools, and you can write a relatively small function to handle rounding and display.

5. Mass conservation checker

This is a neat engineering-related addition.

After calculating the reaction, show the total mass of reactants consumed versus the total mass of products formed.

For example:

$$2H_2 + O_2 \rightarrow 2H_2O$$

Substance	Mass
H_2 consumed	4.03 g
O_2 consumed	31.75 g
H_2O produced	35.78 g

Then display:

Mass is conserved!

Why it's achievable: You already calculate reactant and product masses, so this is mostly adding the values and comparing them.

6. My recommended feature combination

If I were building this as an introductory Python project, I'd prioritize these:

Feature	Difficulty	Value
Step-by-step solutions	Easy–Medium	Very high
Reaction difficulty selector	Easy	High
Limiting reactant visualizer	Medium	Very high
Significant figures assistant	Easy	High
Mass conservation checker	Easy	High

My recommendation is to build the following combination:

- Main calculator: Balances equations and calculates theoretical yield.
- Step-by-step mode: Explains all calculations.
- Limiting reactant visualizer: Shows which reactant runs out first.
- Mass conservation checker: Verifies the result.

That would be a pretty impressive introductory Python project without requiring advanced programming.

One question that would help me guide you:

What are you planning to use to build the web app?

1. What framework are you using?

Streamlit (Python)
Flask (Python)
Django (Python)
HTML/CSS/JavaScript with Python backend
I haven't decided yet

2. How comfortable are you with Python?

Beginner — loops, variables, functions
Intermediate — dictionaries, lists, classes
Comfortable with libraries and more complex logic
Submit

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.